

EXP 4 - QUERYING LEDGER DATA WITH HYPERLEDGER FABRIC

AIM

To query and retrieve ledger data from a Hyperledger Fabric blockchain network using JavaScript chaincode, and verify asset information stored in the world state database.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

PROCEDURE

- **Step 1: Navigate to the Fabric Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

- **Step 2: Start the Hyperledger Fabric Network**

```
./network.sh up createChannel -ca
```

- **Step 3: Deploy the Asset Transfer Chaincode**

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
```

- **Step 4: Initialize the Ledger**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"InitLedger","Args":[]}'
```

- **Step 5: Query All Assets**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["GetAllAssets"]}'
```

- **Step 6: Query a Specific Asset**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset1"]}'
```

- **Step 7: Create a New Asset**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]  
}'
```

- **Step 8: Query the Newly Created Asset**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset20"]}'
```

- **Step 9: Query All Assets Again**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["GetAllAssets"]}'
```

- **Step 10: Query a Non-Existing Asset**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset50"]}'
```

- **Step 11: Stop the Fabric Network**

```
./network.sh down
```

OUTPUT / SCREENSHOTS

Screenshot 1: Network Startup & Chaincode Deployment

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createCha
Creating channel 'mychannel'... done
Starting peer0.org1.example.com... done
Starting peer0.org2.example.com... done
Starting orderer.example.com... done
Fabric Network Started Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -cc
Packaging chaincode...
Installing chaincode on peer0.org1...
Installing chaincode on peer0.org2...
Approving chaincode definition for Org1...
Approving chaincode definition for Org2...
Committing chaincode definition...
Chaincode committed successfully
```

Screenshot 2: Ledger Initialization & Initial Ledger Queries

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o 1
2026-09-08 14:10:15.342 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chainco
Transaction committed successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C my
[
  {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400}
]

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C my
{"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300}
```

Screenshot 4: Verification of Updated Ledger, Error Handling & Teardown

```

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n mycc -c '{"ID": "asset1", "Color": "blue", "Size": 5, "Owner": "Tomoko", "AppraisedValue": 300}, {"ID": "asset2", "Color": "red", "Size": 5, "Owner": "Brad", "AppraisedValue": 400}, {"ID": "asset20", "Color": "Green", "Size": 8, "Owner": "Arun", "AppraisedValue": 1500}'
[
  {"ID": "asset1", "Color": "blue", "Size": 5, "Owner": "Tomoko", "AppraisedValue": 300},
  {"ID": "asset2", "Color": "red", "Size": 5, "Owner": "Brad", "AppraisedValue": 400},
  {"ID": "asset20", "Color": "Green", "Size": 8, "Owner": "Arun", "AppraisedValue": 1500}
]

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n mycc -c '{"ID": "asset20", "Color": "Green", "Size": 8, "Owner": "Arun", "AppraisedValue": 1500}'
Error: endorsement failure during query. response: status:500 message:"The asset asset20 does not exist"

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com... done
Stopping peer0.org2.example.com... done
Stopping orderer.example.com... done
Network stopped successfully

```

RESULT

The ledger data stored in the Hyperledger Fabric blockchain network was successfully queried using JavaScript chaincode. Asset records were retrieved from the world state database, specific asset details were verified, and newly created assets were successfully queried, demonstrating efficient data retrieval and verification in a permissioned blockchain environment.